

1. Executive Summary

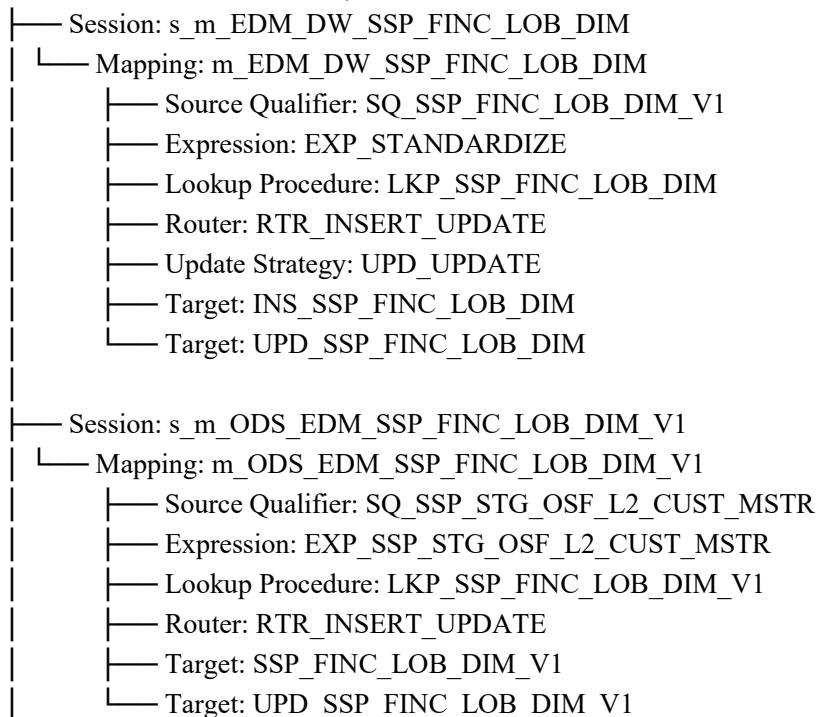
This document describes the analysis and migration of Wkfl_SSP_FINC_LOB_DIM, an Informatica PowerCenter workflow responsible for maintaining the Financial Line of Business (LOB) Dimension table (SSP_FINC_LOB_DIM) in the data warehouse. The workflow propagates LOB reference data across three data layers — ODS/Staging, EDM (Enterprise Data Model), and DW (Data Warehouse) — through two sequentially executed sessions.

Session 1 (s_m_ODS_EDM_SSP_FINC_LOB_DIM_V1) reads new LOB identifiers (FNCL_LOB_ID) detected in the ODS staging table (SSP_STG.SSP_STG_OSF_L2_CUST_MSTR) and bootstraps placeholder dimension records into the EDM intermediate table (SSP_FINC_LOB_DIM_V1) with default constant values. Only one column originates from the source — all remaining attributes are assigned hardcoded defaults (e.g., 'UNKNOWN', 'ARBOR', 'ODS') at the Expression transformation layer, pending enrichment from authoritative upstream systems.

Session 2 (s_m_EDM_DW_SSP_FINC_LOB_DIM) reads the enriched and fully attributed records from EDM.SSP_FINC_LOB_DIM_V1 — the output of Session 1 — and promotes them into the final DW dimension table (SSP_FINC_LOB_DIM). Both sessions implement an insert-or-update pattern using a Lookup + Router combination: a lookup determines whether the LOB already exists in the target, and a Router directs new records to insert and existing records to update.

2. Workflow Overview

Wkfl_SSP_FINC_LOB_DIM |



** The graph model is missing CMD tasks so they don't show up here

3. s_m_ODS_EDM_SSP_FINC_LOB_DIM_V1 Narrative

Step 1: Read Source Data

Transformation name: SQ_SSP_STG_OSF_L2_CUST_MSTR

The workflow reads customer records from SSP_STG.SSP_STG_OSF_L2_CUST_MSTR.

Retrieved attributes include:

- FNCL_LOB_ID

Step 2: Expression

Transformation: EXP_SSP_STG_OSF_L2_CUST_MSTR

Expressions:

Expression list (column: expression)

- **IsCurrent:** 1
- **SRCSYSID:** 1
- **FINC_LOB_ARBR_CD:** 'ARBOR'
- **FINC_LOB_NM:** 'UNKNOWN'
- **FINC_LOB_BUS_NM:** 'UNKNOWN'
- **FINC_LOB_BUS_NM_UAT:** 'UNKNOWN'
- **FINC_LOB_HRCHY_ID:** 'UNKNOWN'
- **ACTV_DT:** SESSSTARTTIME
- **INACTV_DT:** "TO_DATE('31-DEC-9999', 'DD-MON-YYYY')"
- **IS_ACTV_IND:** 'Y'
- **DIM_SRC_CD:** 'ODS'
- **DIM_LAST_MDFD_DT:** SESSSTARTTIME
- **FINC_LOB_HRCHY_LVL:** 'UNKNOWN'

Input columns

- FNCL_LOB_ID

Output columns

- FNCL_LOB_ID
- IsCurrent
- SRCSYSID
- FINC_LOB_ARBR_CD
- FINC_LOB_NM
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_HRCHY_ID
- ACTV_DT
- INACTV_DT
- IS_ACTV_IND
- DIM_SRC_CD

- DIM_LAST_MDFD_DT
- FINC_LOB_HRCHY_LVL

Step 3: Reference Data Enrichment (Lookup)

Transformation: LKP_SSP_FINC_LOB_DIM

Join condition: FINC_LOB_CD = FNCL_LOB_ID

tableName: SSP_FINC_LOB_DIM_V1

Input columns:

- FNCL_LOB_ID

Output columns:

- SSP_FINC_LOB_DIM_Key

Step 4: Data Filtering (Router)

Transformation: RTR_INSERT_UPDATE

Groups (name: condition)

- **Input:** N/A
- **INSGRP:** ISNULL(SSP_FINC_LOB_CD_DIM_Key)
- **UPDGRP:** IIF(ISNULL(SSP_FINC_LOB_CD_DIM_Key),TRUE,FALSE)
- **DEFAULT1:** N/A

Input/Output columns:

- FNCL_LOB_ID
- IsCurrent
- SRCSYSID
- FINC_LOB_ARBR_CD
- FINC_LOB_NM
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_HRCHY_ID
- ACTV_DT
- INACTV_DT
- IS_ACTV_IND
- DIM_SRC_CD
- DIM_LAST_MDFD_DT
- FINC_LOB_HRCHY_LVL
- SSP_FINC_LOB_DIM_Key

Step 4: Record Updates

Transformation: UPD_SSP_FINC_LOB_DIM

Target: SSP_FINC_LOB_DIM

Settings:

- Tracing level: normal
- Update Strategy Expression: DD_UPDATE
- Forward Rejected Rows: yes

Input columns:

- FNCL_LOB_ID
- IsCurrent
- SRCSYSID
- FINC_LOB_ARBR_CD
- FINC_LOB_NM
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_HRCHY_ID
- ACTV_DT
- INACTV_DT
- IS_ACTV_IND
- DIM_SRC_CD
- DIM_LAST_MDFD_DT
- FINC_LOB_HRCHY_LVL
- SSP_FINC_LOB_DIM_Key

Output columns:

- IsCurrent
- SRCSYSID
- FINC_LOB_ARBR_CD
- FINC_LOB_NM
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_HRCHY_ID
- ACTV_DT
- INACTV_DT
- IS_ACTV_IND
- DIM_SRC_CD
- DIM_LAST_MDFD_DT
- FINC_LOB_HRCHY_LVL
- SSP_FINC_LOB_DIM_Key

Step 5: Record Inserts

Transformation: INS_SSP_FINC_LOB_DIM

Target: SSP_FINC_LOB_DIM

Input columns:

- FNCL_LOB_ID
- IsCurrent
- SRCSYSID
- FINC_LOB_ARBR_CD
- FINC_LOB_NM
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_HRCHY_ID
- ACTV_DT
- INACTV_DT
- IS_ACTV_IND
- DIM_SRC_CD
- DIM_LAST_MDFD_DT
- FINC_LOB_HRCHY_LVL
- SSP_FINC_LOB_DIM_Key

Output columns:

- FNCL_LOB_ID
- IsCurrent
- SRCSYSID
- FINC_LOB_ARBR_CD
- FINC_LOB_NM
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_HRCHY_ID
- ACTV_DT
- INACTV_DT
- IS_ACTV_IND
- DIM_SRC_CD
- DIM_LAST_MDFD_DT
- FINC_LOB_HRCHY_LVL

4. s_m_EDM_DW_SSP_FINC_LOB_DIM Narrative

Step 1: Read Source Data

Transformation name: SQ_SSP_FINC_LOB_DIM_V1

The workflow reads records from EDM.SSP_FINC_LOB_DIM_V1.

Retrieved attributes include:

- ACTV_DT
- DIM_LAST_MDFD_DT
- DIM_SRC_CD
- EndDate
- FINC_LOB_ARBR_CD
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_CD
- FINC_LOB_HRCHY_ID
- FINC_LOB_HRCHY_LVL1
- FINC_LOB_HRCHY_LVL2
- FINC_LOB_HRCHY_LVL3
- FINC_LOB_NM
- INACTV_DT
- IS_ACTV_IND
- IsCurrent
- SSP_FINC_LOB_DIM_Key
- SourceSystemID
- SourceSystemKey
- StartDate

Step 2: Reference Data Enrichment (Lookup)

Transformation: LKP_SSP_FINC_LOB_DIM

Join condition: FINC_LOB_CD = IN_FINC_LOB_CD

tableName: SSP_FINC_LOB_DIM

Input columns:

- IN_FINC_LOB_CD

Output columns:

- FINC_LOB_CD_SK

Step 3: Data Filtering (Router)

Transformation: RTR_INSERT_UPDATE

Transformation: RTR_INSERT_UPDATE

Groups (name: condition)

- **Input:** N/A
- **INSGRP:** ISNULL(FINC_LOB_CD_SK)
- **UPDGRP:** IIF(NOT ISNULL(FINC_LOB_CD_SK),TRUE,FALSE)
- **DEFAULT1:** N/A

Input/Output columns:

- ACTV_DT
- DIM_LAST_MDFD_DT
- DIM_SRC_CD
- EndDate
- FINC_LOB_ARBR_CD
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_CD
- FINC_LOB_HRCHY_ID
- FINC_LOB_HRCHY_LVL1
- FINC_LOB_HRCHY_LVL2
- FINC_LOB_HRCHY_LVL3
- FINC_LOB_NM
- INACTV_DT
- IS_ACTV_IND
- IsCurrent
- SSP_FINC_LOB_DIM_Key
- SourceSystemID
- SourceSystemKey
- StartDate
- FINC_LOB_CD_SK

Step 4: Record Updates

Transformation: UPD_SSP_FINC_LOB_DIM

Target: SSP_FINC_LOB_DIM

Settings:

- Tracing level: normal
- Update Strategy Expression: DD_UPDATE
- Forward Rejected Rows: yes

Input columns:

- ACTV_DT
- DIM_LAST_MDFD_DT
- DIM_SRC_CD
- EndDate
- FINC_LOB_ARBR_CD
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_CD
- FINC_LOB_HRCHY_ID
- FINC_LOB_HRCHY_LVL1
- FINC_LOB_HRCHY_LVL2
- FINC_LOB_HRCHY_LVL3
- FINC_LOB_NM
- INACTV_DT
- IS_ACTV_IND
- IsCurrent
- SSP_FINC_LOB_DIM_Key
- SourceSystemID
- SourceSystemKey
- StartDate

Output columns:

- ACTV_DT
- DIM_LAST_MDFD_DT
- DIM_SRC_CD
- FINC_LOB_ARBR_CD
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_CD
- FINC_LOB_HRCHY_ID
- FINC_LOB_HRCHY_LVL1
- FINC_LOB_HRCHY_LVL2
- FINC_LOB_HRCHY_LVL3
- FINC_LOB_NM
- INACTV_DT
- IS_ACTV_IND

Step 5: Record Inserts

Transformation: INS_SSP_FINC_LOB_DIM

Target: SSP_FINC_LOB_DIM

Input columns:

- ACTV_DT
- DIM_LAST_MDFD_DT
- DIM_SRC_CD
- EndDate
- FINC_LOB_ARBR_CD
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- FINC_LOB_CD
- FINC_LOB_HRCHY_ID
- FINC_LOB_HRCHY_LVL1
- FINC_LOB_HRCHY_LVL2
- FINC_LOB_HRCHY_LVL3
- FINC_LOB_NM
- INACTV_DT
- IS_ACTV_IND
- IsCurrent
- SSP_FINC_LOB_DIM_Key
- SourceSystemID
- SourceSystemKey
- StartDate

Output columns:

- FINC_LOB_HRCHY_ID
- INACTV_DT
- ACTV_DT
- FINC_LOB_HRCHY_LVL1
- FINC_LOB_HRCHY_LVL2
- FINC_LOB_HRCHY_LVL3
- DIM_SRC_CD
- FINC_LOB_CD
- IS_ACTV_IND
- SSP_FINC_LOB_DIM_Key
- FINC_LOB_BUS_NM
- FINC_LOB_BUS_NM_UAT
- DIM_LAST_MDFD_DT
- FINC_LOB_HRCHY_ID
- FINC_LOB_NM
- FINC_LOB_ARBR_CD

4. Business Rules Extracted

Default values (column: value):

- **SRCSYSID:** 1
- **FINC_LOB_ARBR_CD:** 'ARBOR'
- **FINC_LOB_NM:** 'UNKNOWN'
- **FINC_LOB_BUS_NM:** 'UNKNOWN'
- **FINC_LOB_BUS_NM_UAT:** 'UNKNOWN'
- **FINC_LOB_HRCHY_ID:** 'UNKNOWN'
- **ACTV_DT:** SESSSTARTTIME
- **INACTV_DT:** "TO_DATE('31-DEC-9999', 'DD-MON-YYYY')"
- **IS_ACTV_IND:** 'Y'
- **DIM_SRC_CD:** 'ODS'
- **DIM_LAST_MDFD_DT:** SESSSTARTTIME
- **FINC_LOB_HRCHY_LVL:** 'UNKNOWN'

5. Source-to-Target Mapping

5A. Mapping: m_ODS_EDM_SSP_FINC_LOB_DIM_V1

- **Source:** SSP_STG.SSP_STG_OSF_L2_CUST_MSTR
- **Target:** EDM.SSP_FINC_LOB_DIM_V1

SI No	Source Column	Source Table	Transformation Path	Expression / Logic	Origin Type	Target Column	Target Table
1	FNCL_LOB_ID	SSP_STG_OSF_L2_CUST_MSTR	SQ → EXP → RTR → Target	Direct pass-through	Direct	FNCL_LOB_ID	SSP_FINC_LOB_DIM_V1
2	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR		1 Constant	IsCurrent	SSP_FINC_LOB_DIM_V1
3	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR		1 Constant	SRCSYSID	SSP_FINC_LOB_DIM_V1
4	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	ARBOR'	Constant	FINC_LOB_ARBR_CD	SSP_FINC_LOB_DIM_V1
5	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	UNKNOWN'	Constant	FINC_LOB_NM	SSP_FINC_LOB_DIM_V1
6	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	UNKNOWN'	Constant	FINC_LOB_BUS_NM	SSP_FINC_LOB_DIM_V1
7	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	UNKNOWN'	Constant	FINC_LOB_BUS_NM_UAT	SSP_FINC_LOB_DIM_V1
8	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	UNKNOWN'	Constant	FINC_LOB_HRCHY_ID	SSP_FINC_LOB_DIM_V1
9	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	SESSSTARTTIME	Session Var	ACTV_DT	SSP_FINC_LOB_DIM_V1
10	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	TO_DATE('31-DEC-9999','DD-MON-YYYY')	Constant	INACTV_DT	SSP_FINC_LOB_DIM_V1
11	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	Y'	Constant	IS_ACTV_IND	SSP_FINC_LOB_DIM_V1
12	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	ODS'	Constant	DIM_SRC_CD	SSP_FINC_LOB_DIM_V1
13	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	SESSSTARTTIME	Session Var	DIM_LAST_MDFD_DT	SSP_FINC_LOB_DIM_V1
14	—	—	EXP_SSP_STG_OSF_L2_CUST_MSTR	UNKNOWN'	Constant	FINC_LOB_HRCHY_LVL	SSP_FINC_LOB_DIM_V1
15	FNCL_LOB_ID	SSP_STG_OSF_L2_CUST_MSTR	SQ → LKP_SSP_FINC_LOB_DIM_V1	Lookup: FINC_LOB_CD = FNCL_L	Lookup	SSP_FINC_LOB_DIM_Key	SSP_FINC_LOB_DIM_V1

Routing logic (RTR_INSERT_UPDATE):

INSGRP:

ISNULL(SSP_FINC_LOB_DIM_Key) → new record, goes to insert target SSP_FINC_LOB_DIM_V1

UPDGRP:

NOT ISNULL(SSP_FINC_LOB_DIM_Key) → existing record, goes to update target
UPD_SSP_FINC_LOB_DIM_V1

Note: This mapping is a seed/bootstrap mapping. Only FNCL_LOB_ID originates from the source table. All 13 remaining columns are either hardcoded constants or session-time variables, indicating

this mapping creates placeholder LOB dimension records with default values when a new LOB ID is detected.

5B. Mapping: m_EDM_DW_SSP_FINC_LOB_DIM

- **Source:** EDM.SSP_FINC_LOB_DIM_V1
- **Target:** DW.SSP_FINC_LOB_DIM

SI No	Source Column	Source Table	Transformation Path	Expression / Logic	Origin Type	Target Column	Target Table
1	ACTV_DT	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	ACTV_DT	SSP_FINC_LOB_DIM
2	DIM_LAST_MDFD_DT	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	DIM_LAST_MDFD_DT	SSP_FINC_LOB_DIM
3	DIM_SRC_CD	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	DIM_SRC_CD	SSP_FINC_LOB_DIM
4	EndDate	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	EndDate	SSP_FINC_LOB_DIM
5	FINC_LOB_ARBR_CD	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	FINC_LOB_ARBR_CD	SSP_FINC_LOB_DIM
6	FINC_LOB_BUS_NM	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	FINC_LOB_BUS_NM	SSP_FINC_LOB_DIM
7	FINC_LOB_BUS_NM_UAT	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	FINC_LOB_BUS_NM_UAT	SSP_FINC_LOB_DIM
8	FINC_LOB_CD	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	FINC_LOB_CD	SSP_FINC_LOB_DIM
9	FINC_LOB_HRCHY_ID	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	FINC_LOB_HRCHY_ID	SSP_FINC_LOB_DIM
10	FINC_LOB_HRCHY_LVL1	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	FINC_LOB_HRCHY_LVL1	SSP_FINC_LOB_DIM
11	FINC_LOB_HRCHY_LVL2	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	FINC_LOB_HRCHY_LVL2	SSP_FINC_LOB_DIM
12	FINC_LOB_HRCHY_LVL3	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	FINC_LOB_HRCHY_LVL3	SSP_FINC_LOB_DIM
13	FINC_LOB_NM	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	FINC_LOB_NM	SSP_FINC_LOB_DIM
14	INACTV_DT	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	INACTV_DT	SSP_FINC_LOB_DIM
15	IS_ACTV_IND	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	IS_ACTV_IND	SSP_FINC_LOB_DIM
16	IsCurrent	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	IsCurrent	SSP_FINC_LOB_DIM
17	SSP_FINC_LOB_DIM_Key	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	SSP_FINC_LOB_DIM_Key	SSP_FINC_LOB_DIM
18	SourceSystemID	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	SourceSystemID	SSP_FINC_LOB_DIM
19	SourceSystemKey	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	SourceSystemKey	SSP_FINC_LOB_DIM
20	StartDate	EDM.SSP_FINC_LOB_DIM_V1	SQ → RTR → INS / UPD	Direct pass-through	Direct	StartDate	SSP_FINC_LOB_DIM
21	FINC_LOB_CD	EDM.SSP_FINC_LOB_DIM_V1	SQ → LKP_SSP_FINC_LOB_DIM	FINC_LOB_CD = IN_FINC_LOB_CD on SSP_FINC_LOB_DIM → returns FINC_LOB_CD_SK	Lookup	FINC_LOB_CD_SK *(routing key only)	—

Routing logic (RTR_INSERT_UPDATE):

INSGRP:

ISNULL(FINC_LOB_CD_SK) → new record, goes to insert target INS_SSP_FINC_LOB_DIM

UPDGRP:

NOT ISNULL(FINC_LOB_CD_SK) → existing record, goes to update target
UPD_SSP_FINC_LOB_DIM (strategy: DD_UPDATE)

Neo4j Cypher Queries to Generate This Mapping

- **Query 1 — Full field-level source-to-target column lineage:**

```

MATCH (srcField:Column:Field)-[:BOUND_TO_PORT]->(srcPort:Column:Port)
-[:FLOWS_TO*1..10]->(tgtPort:Column:Port)-[:BOUND_TO_FIELD]->(tgtField:Column:Field)
MATCH (srcT:Transformation)-[:HAS_PORT]->(srcPort)
MATCH (tgtT:Transformation)-[:HAS_PORT]->(tgtPort)
RETURN

```

```

srcField.name      AS source_column,
srcT.name           AS source_transformation,
tgtT.name           AS target_transformation,
tgtField.name       AS target_column

```

ORDER BY source_column

- **Query 2 — Get all expressions applied per port (for the Expression transformation):**

```
MATCH      (m:Mapping)-[:HAS_TRANSFORMATION]->(t:Transformation)-[:HAS_PORT]-
>(p:Column:Port)

-[:USES_EXPRESSION]->(e:Expression)

WHERE m.name IN [

    'm_ODS_EDM_SSP_FINC_LOB_DIM_V1',

    'm_EDM_DW_SSP_FINC_LOB_DIM'

]

RETURN

    m.name      AS mapping,
    t.name      AS transformation,
    p.name      AS port_name,
    p.portType  AS port_type,
    e.rawExpression  AS expression,
    e.intent    AS intent

ORDER BY m.name, t.name, p.ordinalPosition
```

- **Query 3 — Get lookup conditions (join key and lookup table):**

```
MATCH      (m:Mapping)-[:HAS_TRANSFORMATION]->(t:Transformation)-[:HAS_LOOKUP]-
>(lkp:LookupCondition)

WHERE m.name IN [

    'm_ODS_EDM_SSP_FINC_LOB_DIM_V1',

    'm_EDM_DW_SSP_FINC_LOB_DIM'

]

RETURN

    m.name      AS mapping,
    t.name      AS lookup_transformation,
    lkp.tableName  AS lookup_table,
    lkp.joinCondition  AS join_condition
```

- **Query 4 — Get router group conditions:**

```

MATCH (m:Mapping)-[:HAS_TRANSFORMATION]->(t:Transformation)-[:HAS_GROUP]->(rg:RouterGroup)

WHERE m.name IN [
    'm_ODS_EDM_SSP_FINC_LOB_DIM_V1',
    'm_EDM_DW_SSP_FINC_LOB_DIM'
]

RETURN
    m.name          AS mapping,
    t.name           AS router_transformation,
    rg.name          AS group_name,
    rg.condition      AS routing_condition

ORDER BY m.name, rg.name

```

- **Query 5 — Full mapping summary: source field → transformation chain → target field:**

```

MATCH (m:Mapping)-[:HAS_TRANSFORMATION]->(t:Transformation)

WHERE m.name IN [
    'm_ODS_EDM_SSP_FINC_LOB_DIM_V1',
    'm_EDM_DW_SSP_FINC_LOB_DIM'
]

MATCH (t)-[:HAS_PORT]->(p:Column:Port)

OPTIONAL MATCH (p)-[:USES_EXPRESSION]->(e:Expression)

OPTIONAL MATCH (srcField:Column:Field)-[:BOUND_TO_PORT]->(p)

OPTIONAL MATCH (p)-[:BOUND_TO_FIELD]->(tgtField:Column:Field)

RETURN
    m.name          AS mapping,
    t.name           AS transformation,
    t.type           AS transformation_type,
    p.name           AS port_name,
    p.portType       AS port_type,
    e.rawExpression   AS expression,
    srcField.name     AS bound_source_field,

```

tgtField.name AS bound_target_field

ORDER BY m.name, t.name, p.ordinalPosition

6. Lineage Summary

[Section 6 – Lineage Flow Diagram | Wkfl_SSP_FINC_LOB_DIM](#)

This workflow (Wkfl_SSP_FINC_LOB_DIM) moves Financial LOB dimension data across three data layers in two sequential sessions. Session 1 feeds Session 2 — the output table of Session 1 (EDM.SSP_FINC_LOB_DIM_V1) is the direct source of Session 2.]

Session 1 — m_ODS_EDM_SSP_FINC_LOB_DIM_V1

- **Layer:** ODS/Staging → EDM (Intermediate Dimension)

Step	Transformation	What Happens
1	SQ_SSP_STG_OSF_L2_CUST_MSTR	Reads FNCL_LOB_ID from SSP_STG.SSP_STG_OSF_L2_CUST_MSTR
2	EXP_SSP_STG_OSF_L2_CUST_MSTR	Passes FNCL_LOB_ID through; generates 13 additional columns with constant default values (UNKNOWN, ARBOR, session timestamps, Y, ODS)
3	LKP_SSP_FINC_LOB_DIM_V1	Looks up SSP_FINC_LOB_DIM_Key in EDM.SSP_FINC_LOB_DIM_V1 using FNCL_LOB_ID
4	RTR_INSERT_UPDATE	Routes record: NULL key → INSERT; existing key → UPDATE
5	Target	Writes to EDM.SSP_FINC_LOB_DIM_V1 (insert or update)

- **Summary:**

SSP_STG.SSP_STG_OSF_L2_CUST_MSTR

↓ (FNCL_LOB_ID)

EXP_SSP_STG_OSF_L2_CUST_MSTR ← constant defaults applied here

↓

LKP_SSP_FINC_LOB_DIM_V1 ← checks EDM.SSP_FINC_LOB_DIM_V1

↓

RTR_INSERT_UPDATE

└─ INSERT → EDM.SSP_FINC_LOB_DIM_V1 (new LOB ID)

└─ UPDATE → EDM.SSP_FINC_LOB_DIM_V1 (existing LOB ID)

Session 2 — m_EDM_DW_SSP_FINC_LOB_DIM

- **Layer:** EDM (Intermediate Dimension) → DW (Final Dimension)

Step	Transformation	What Happens
1	SQ_SSP_FINC_LOB_DIM_V1	Reads all 20 enriched columns from EDM.SSP_FINC_LOB_DIM_V1
2	EXP_STANDARDIZE	Standardizes/transforms attribute values
3	LKP_SSP_FINC_LOB_DIM	Looks up FINC_LOB_CD_SK in DW.SSP_FINC_LOB_DIM using FINC_LOB_CD
4	RTR_INSERT_UPDATE	Routes record: NULL key → INSERT; existing key → UPDATE
5	UPD_UPDATE (DD_UPDATE)	Applies update strategy for existing records
6	Target	Writes to DW.SSP_FINC_LOB_DIM (insert or update)

- **Summary:**

EDM.SSP_FINC_LOB_DIM_V1 ← output of Session 1

↓ (all 20 columns)

EXP_STANDARDIZE

↓

LKP_SSP_FINC_LOB_DIM ← checks DW.SSP_FINC_LOB_DIM

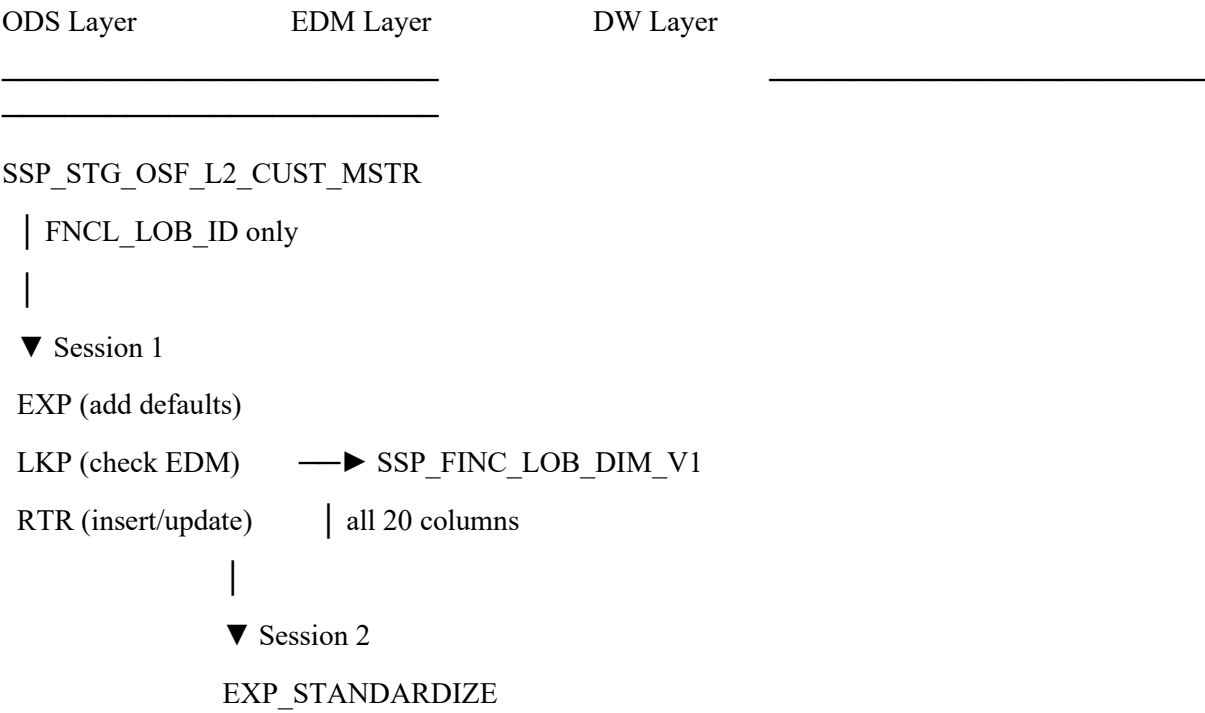
↓

RTR_INSERT_UPDATE

└─ INSERT → INS_SSP_FINC_LOB_DIM → DW.SSP_FINC_LOB_DIM (new record)

└─ UPDATE → UPD_UPDATE (DD_UPDATE) → DW.SSP_FINC_LOB_DIM (existing record)

End-to-End Lineage



LKP (check DW) —► SSP_FINC_LOB_DIM

RTR (insert/update) (final dimension)

Key inter-mapping dependency: EDM.SSP_FINC_LOB_DIM_V1 is both the **write target** of Session 1 and the **read source** of Session 2. Session 1 must complete successfully before Session 2 executes. This table acts as the staging bridge between the ODS and DW layers.

7. Assumptions and Context Layer Insights

7.1 Conversion Framework

- Each Informatica mapping is extracted as a self-contained JSON subgraph by kg_extractor.py and passed to converter.py, which assembles the system prompt + subgraph payload and calls the API.
- The converter targets **Google BigQuery SQL** using a **CTE (WITH clause) chaining pattern** — each transformation stage in Informatica becomes a named CTE (cte_SQ_, cte_EXP_, cte_LKP_, etc.) in the output SQL.
- The **Router** + **Update Strategy** combination is collapsed into a single BigQuery MERGE statement. There is no direct equivalent of Informatica's Router in BigQuery. The routing conditions become the WHEN MATCHED / WHEN NOT MATCHED predicates of the MERGE.
- All workflows are treated as **on-demand** (no schedule). Generated Airflow DAGs default to schedule=None.
- BigQuery table references use parameterized placeholders: '@{PROJECT_ID}.@{DATASET}.<table_name>' and these must be resolved before deployment.

7.2 Function Translation Assumptions

The following Informatica expression functions are assumed to have direct BigQuery equivalents, as defined in the context_layer.function_map:

Informatica Function	BigQuery Equivalent	Notes
IIF(cond, a, b)	IF(cond, a, b)	Direct equivalent
ISNULL(x)	x IS NULL	Predicate, not a function
DECODE(...)	CASE WHEN ... THEN ... ELSE ... END	Structural rewrite
NVL(x, fallback)	COALESCE(x, fallback)	Direct equivalent
NVL2(x, a, b)	IF(x IS NOT NULL, a, b)	Conditional rewrite
TO_DATE('31-DEC-9999','DD-MON-YYYY')	DATE '9999-12-31'	Used as the "far future" inactive date
SESSSTARTTIME	CURRENT_TIMESTAMP()	Session start time → BigQuery wall clock
SYSDATE	CURRENT_DATETIME()	Oracle/Informatica system date
TO_CHAR(d, fmt)	FORMAT_DATETIME(fmt, d)	Format string may need adjustment
DATE_DIFF(a, b, 'MM')	TIMESTAMP_DIFF(a, b, MONTH)	Unit mapping required
ADD_MONTHS(d, n)	DATE_ADD(d, INTERVAL n MONTH)	Direct equivalent
LAST_DAY(d)	DATE_SUB(DATE_TRUNC(DATE_ADD(d, INTERVAL 1 MONTH), MONTH), INTERVAL 1 DAY)	No native BigQuery equivalent
INSTR(s, sub)	STRPOS(s, sub)	Direct equivalent
TRUNC(d)	DATE_TRUNC(d, MONTH) or TRUNC(x)	Context-dependent

Key assumption: All CURRENT_TIMESTAMP() and CURRENT_DATETIME() calls are wrapped as TIMESTAMP(expr, 'America/New_York') to enforce timezone consistency. Informatica SESSSTARTTIME is timezone-naive; this wrapping is an explicit assumption about the operational timezone of the target environment.

7.3 Data Type Mapping Assumptions

All Informatica/Teradata/SQL Server data types are mapped to BigQuery types as follows:

Source Type	BigQuery Type
string, nstring, char, nchar, varchar, nvarchar, text	STRING
integer, int, smallint, bigint	INT64
number, decimal	NUMERIC
float, double, real	FLOAT64
date/time, timestamp	TIMESTAMP
date	DATE
time	TIME
binary	BYTES
bit	BOOL

Assumption: All string-family types are unified into BigQuery STRING. No distinction is made between fixed-length (CHAR) and variable-length (VARCHAR) types, as BigQuery does not have this distinction.

7.4 Transformation Pattern Assumptions

Informatica Transformation	Conversion Pattern Assumed
Source Qualifier	Becomes cte_SQ_* — a plain SELECT from BigQuery table. If sql_override is present, it is used as the CTE body directly.
Expression	Becomes cte_EXP_* — OUTPUT ports become computed SELECT aliases; INPUT/OUTPUT ports are pass-throughs; LOCAL VARIABLE ports become intermediate CTE columns.
Lookup Procedure	Becomes cte_LKP_* — implemented as a LEFT JOIN on the lookup table. A NULL result means no match (new record).
Router	Becomes filter conditions inside the MERGE statement. INSGRP → WHEN NOT MATCHED; UPDGRP → WHEN MATCHED.
Update Strategy (DD_UPDATE)	Collapsed into the WHEN MATCHED THEN UPDATE SET clause of the MERGE.
Update Strategy (DD_INSERT)	Becomes a standalone INSERT INTO statement.
Sequence Generator	NEXTVAL → ROW_NUMBER() OVER (ORDER BY (SELECT NULL)). No Informatica-style persistent sequence is assumed.